

AI Forward Deployed Engineer: Focused Six-Project Program

Superseded planning artifact. The normative candidate documents are `fde-focused-6-projects-opensource.md` and `fde-focused-6-projects-localstack.md`. This ASCII tree is retained only as review history and must not be used for authoring, workload, tooling, GPU, or assessment decisions.

This proposal restructures the AI FDE program into six sequential enterprise engagements. The program remains project-first, includes just-in-time theory inside each task estimate, and stays within **22 weeks x 20 hours/week = 440 total hours**.

```
AI FORWARD DEPLOYED ENGINEER - 6 PROJECTS / 22 WEEKS / 440 HOURS
|
|-- Program delivery contract
|   |-- Projects 1-5 run and are graded locally with Docker Compose.
|   |-- Project 6 first passes local acceptance, then deploys one temporary protected AWS endpoint.
|   |-- No cloud deployment or public endpoint is permitted in Projects 1-5, including Project 1 demonstrations.
|   |-- Every task estimate includes JIT theory, implementation, documentation, review, and defense time.
|   |-- Every project includes a client-facing decision or communication artifact.
|   |-- Customer simulations are asynchronous and deterministic: role cards, timed information injects, evidence templates,
|       |-- recorded defenses, and rubric anchors are published before assessment.
|   |-- Reviewer scoring evaluates the student's evidence, decision quality, and communication against the same scenario,
|       |-- rather than a live stakeholder's improvisation.
|   |-- REST, Git, Python, relational database, and basic cloud competence are verified at admission rather than retaught.
|   |-- Delivery rubrics classify required, supporting, optional, positioning, and AI-specific work without changing the
|       |-- program's local-first grading boundary.
|
+-- Project 1: Client Discovery and System Diagnostics
|   |-- Schedule: Weeks 1-3 / 60 hours
|   |-- Engagement: Diagnose an unreliable AI-enabled enterprise workflow before proposing a solution.
|   |
|   |-- Task 1.1: Engagement kickoff and stakeholder discovery / 14 hours
|   |   |-- Summary: Establish what the client is trying to achieve before investigating the implementation.
|   |   |-- Trace the stated workflow, users, business outcome, constraints, and known exceptions.
|   |   |-- Conduct a simulated discovery interview in which information is incomplete and priorities conflict.
|   |   |-- Separate reported symptoms, assumptions, and facts requiring technical validation.
|   |   |-- Produce a discovery record with open questions, decision owners, and success criteria.
|   |
|   |-- Task 1.2: Runtime orientation and evidence triage / 10 hours
|   |   |-- Summary: Understand the provided platform quickly enough to investigate one important request path.
|   |   |-- Run the supplied API, worker, PostgreSQL, Redis, and observability stack with Docker Compose.
|   |   |-- Follow one request through code, logs, metrics, and distributed traces.
|   |   |-- Use the supplied architecture map instead of producing a full C4 model from scratch.
|   |   |-- Treat monolith-versus-microservices material as a decision prompt, not a standalone theory block.
|   |
|   |-- Task 1.3: Diagnose and repair the seeded failure / 18 hours
|   |   |-- Summary: Reproduce the client-visible failure and make one evidence-backed repair.
|   |   |-- Run one deterministic load and provider-latency scenario.
|   |   |-- Repair one trace, metric, or dashboard defect selected by the scenario.
|   |   |-- Identify the first supported bottleneck and reject one plausible false explanation.
|   |   |-- Validate the repair using the same scenario and record before-and-after evidence.
|   |
|   |-- Task 1.4: Problem framing and recommendation playback / 10 hours
|   |   |-- Summary: Translate technical evidence into a decision the stakeholder can act on.
|   |   |-- Rewrite the initial problem statement using the verified workflow and evidence.
|   |   |-- Present alternatives including a no-AI or process-change option when appropriate.
|   |   |-- Estimate effort and capacity using named assumptions rather than a full capacity-planning module.
|   |   |-- Deliver separate engineering-detail and business-impact explanations.
|   |
|   |-- Task 1.5: Diagnostic handoff / 8 hours
|   |   |-- Summary: Leave the client with an auditable diagnosis and a bounded next decision.
|   |   |-- Submit the repaired code, evidence packet, concise ADR, and updated workflow trace.
|   |   |-- Record unresolved risks, excluded work, and the next discovery questions.
|   |   |-- Defend ownership of the recommendation when the stakeholder challenges an assumption.
|   |
+-- Project 2: Enterprise Data Integration and Hybrid RAG
|   |-- Schedule: Weeks 4-7 / 80 hours
|   |-- Engagement: Scope and deliver a retrieval system over messy client data without expanding into a platform rewrite.
|   |
|   |-- Task 2.1: Data and use-case discovery / 12 hours
|   |   |-- Summary: Determine whose decision the retrieval system supports and which data can be used safely.
|   |   |-- Interview simulated domain, data-owner, and compliance stakeholders.
|   |   |-- Document source ownership, access constraints, freshness needs, and failure consequences.
|   |   |-- Define measurable retrieval acceptance criteria before choosing chunking or embedding techniques.
|   |   |-- Identify one process or data-quality problem that RAG should not conceal.
|   |
|   |-- Task 2.2: Messy-document ingestion / 20 hours
|   |   |-- Summary: Build a reproducible pipeline for the supplied enterprise document set.
|   |   |-- Extract text and tables from provided PDF and HTML fixtures.
|   |   |-- Normalize metadata, duplicate identifiers, malformed content, and access labels.
|   |   |-- Implement bounded chunking and local embedding using the supplied pipeline scaffold.
|   |   |-- Produce ingestion-quality evidence for published and held-out cases.
|   |
|   |-- Task 2.3: Hybrid retrieval service / 24 hours
|   |   |-- Summary: Implement the smallest production-shaped retrieval boundary that meets the agreed use case.
|   |   |-- Combine PostgreSQL/pgvector dense retrieval with BM25 keyword retrieval.
|   |   |-- Re-rank hybrid candidates with the supplied local cross-encoder before returning the final result set.
|   |   |-- Preserve source metadata and authorization attributes through retrieval.
|   |   |-- Extend a supplied versioned API contract instead of teaching REST fundamentals.
|   |   |-- Use provided migration, repository, and connection-pooling infrastructure.
|   |   |-- Discuss cache freshness and replication only where they affect the selected use case.
|   |
|   |-- Task 2.4: Retrieval evaluation and failure analysis / 14 hours
|   |   |-- Summary: Demonstrate when retrieval succeeds, when it fails, and why.
|   |   |-- Build a compact golden query set with relevance expectations.
|   |   |-- Measure retrieval quality and latency against the agreed acceptance criteria.
|   |   |-- Analyze misses caused by parsing, chunking, metadata, or ranking.
|   |   |-- Recommend one evidence-backed improvement without adding an unbounded RAG feature set.
|   |
|   |-- Task 2.5: Scope and statement-of-work defense / 10 hours
|   |   |-- Summary: Convert discovery and technical findings into a delivery agreement.
|   |   |-- Write a concise SOW covering outcomes, deliverables, assumptions, exclusions, and acceptance criteria.
|   |   |-- Estimate the next delivery increment and identify its largest uncertainty.
|   |   |-- Respond to a stakeholder request that would expand scope without moving time or budget.
|   |   |-- Defend what will not be built and document the resulting decision.
|   |
+-- Project 3: Reliable AI Service Delivery
|   |-- Schedule: Weeks 8-11 / 80 hours
|   |-- Engagement: Stabilize an AI service whose external provider behavior and client priorities change under load.
|   |
|   |-- Task 3.1: Local AI runtime and service boundary / 18 hours
|   |   |-- Summary: Operate the retrieval-enabled platform with one supported local model path and a deterministic provider path.
|   |   |-- Run the supplied multi-service Docker Compose environment.
|   |   |-- Use one provider adapter across local inference and deterministic emulator scenarios.
|   |   |-- Configure the provided vLLM path where hardware supports it or the grading-equivalent Ollama fallback.
|   |   |-- Explain Kubernetes and cloud topology only as alternatives relevant to this service.
|   |
|   |-- Task 3.2: Provider resilience / 20 hours
|   |   |-- Summary: Keep the client workflow safe during latency, rate-limit, malformed-response, and outage scenarios.
|   |   |-- Implement one primary resilience pattern: bounded retry with backoff or circuit breaker with fallback.
|   |   |-- Enforce request deadlines, structured failure responses, and idempotent behavior where required.
|   |   |-- Prove behavior using the deterministic provider scenario pack.
|   |   |-- Treat Kafka, Redpanda, and alternative resilience patterns as optional extensions rather than core work.
|   |
|   |-- Task 3.3: SLO and failure lab / 18 hours
|   |   |-- Summary: Connect one operational objective to a client-visible failure and recovery workflow.
|   |   |-- Define one SLO and one alert tied to user impact.
|   |   |-- Run one reversible failure lab and capture timeline, detection, mitigation, and recovery evidence.
|   |   |-- Validate the selected resilience pattern under failure.
|   |   |-- Document remaining failure modes without building a full chaos suite.
|   |
|   |-- Task 3.4: Changing-requirements simulation / 12 hours
|   |   |-- Summary: Re-plan delivery after a stakeholder changes a reliability requirement mid-engagement.
|   |   |-- Clarify the requested outcome, urgency, decision owner, and hidden trade-off.
|   |   |-- Produce an impact assessment covering scope, risk, schedule, and operational cost.
|   |   |-- Negotiate what moves out when the new requirement moves in.
|   |   |-- Record the approved change instead of silently absorbing it.
|   |
|   |-- Task 3.5: Incident communication and operational handoff / 12 hours
|   |   |-- Summary: Communicate a service failure honestly and leave an executable recovery process.
|   |   |-- Write an engineering incident update and a separate stakeholder status update.
|   |   |-- Run a short incident briefing with uncertain recovery information.
|   |   |-- Deliver a focused runbook for detection, mitigation, rollback, and escalation.
|   |   |-- Convert the failure lesson into one reusable delivery pattern.
|   |
+-- Project 4: Security, Guardrails and Governance Approval
|   |-- Schedule: Weeks 12-15 / 80 hours
|   |-- Engagement: Secure one high-value AI workflow and obtain a defensible stakeholder go/no-go decision.
|   |
|   |-- Task 4.1: Security and governance discovery / 12 hours
|   |   |-- Summary: Identify the workflow's actual actors, sensitive data, decision rights, and risk tolerance.
|   |   |-- Facilitate a simulated workshop with engineering, security, compliance, and business stakeholders.
|   |   |-- Reconcile conflicting availability, privacy, cost, and audit expectations.
|   |   |-- Define which risks require controls, acceptance, transfer, or feature removal.
|   |
|   |-- Task 4.2: Threat model and control selection / 18 hours
|   |   |-- Summary: Model one workflow deeply enough to choose proportionate controls.
|   |   |-- Build one STRIDE-based threat model emphasizing prompt injection and unsafe output handling.
|   |   |-- Map trust boundaries, assets, threat actors, and high-impact abuse cases.
|   |   |-- Select controls using evidence and explicitly reject unnecessary complexity.
|   |   |-- Limit legal-framework coverage to educational engineering control mapping.
|   |
|   |-- Task 4.3: Implement the security boundary / 30 hours
|   |   |-- Summary: Implement and verify the smallest control set needed for the selected workflow.
|   |   |-- Protect one endpoint using the supplied OIDC/RBAC scaffold.
|   |   |-- Add prompt-injection handling, output validation, PII redaction, and audit logging at one AI boundary.
|   |   |-- Configure one CI security gate and one secrets-management improvement.
|   |   |-- Validate controls with deterministic attack and data-leakage scenarios.
|   |   |-- Exercise the selected controls against a second held-out attack scenario and explain any remaining gap.
|   |   |-- Use development identity and secret stores rather than operating production Keycloak or Vault.
|   |
|   |-- Task 4.4: Governance evidence and adversarial review / 12 hours
|   |   |-- Summary: Make the security claim inspectable by people who did not build the system.
|   |   |-- Produce a concise control matrix linking risks, controls, tests, evidence, and owners.
|   |   |-- Review a flawed AI-generated threat or compliance artifact for overclaims and missing controls.
|   |   |-- Record residual risks, evidence limits, and required human oversight.
|   |
|   |-- Task 4.5: Executive risk decision / 8 hours
|   |   |-- Summary: Obtain a go, conditional-go, or no-go decision without hiding technical uncertainty.
|   |   |-- Present the value, material risks, implemented controls, and remaining exposure to a simulated executive.
|   |   |-- Respond to pressure to describe educational evidence as production certification.
|   |   |-- Document the decision, conditions, owner, and review date.
|   |
+-- Project 5: Multi-Agent Workflow Under Ambiguity
|   |-- Schedule: Weeks 16-19 / 80 hours
|   |-- Engagement: Deliver one bounded agent workflow for a stakeholder whose process and priorities evolve during the build.
|   |   |-- Repository relationship: Project 5 creates the shared capstone repository; Project 6 deploys, accepts, and hands
|   |       |-- back that same locally accepted repository. It is one client engagement assessed at two delivery gates.
|   |
|   |-- Task 5.1: Ambiguous client brief and workflow discovery / 12 hours
|   |   |-- Summary: Turn a vague automation request into an observable workflow and decision problem.
|   |   |-- Trace the current process, handoffs, exceptions, failure costs, and human decision points.
|   |   |-- Identify where deterministic software is safer or cheaper than an agent.
|   |   |-- Define the agent's authority boundary, escalation conditions, and prohibited actions.
|   |   |-- Maintain an assumption log as stakeholder information changes.
|   |
|   |-- Task 5.2: Scope and value hypothesis / 10 hours
|   |   |-- Summary: Choose a narrow agent use case with an evidence-based business outcome.
|   |   |-- Define one value hypothesis using revenue, risk, time, or operating cost.
|   |   |-- Estimate delivery effort and operational cost using explicit assumptions.
|   |   |-- Write acceptance criteria, exclusions, rollback conditions, and human-override requirements.
|   |   |-- Defend a smaller scope when the stakeholder requests broad autonomy.
|   |
|   |-- Task 5.3: Bounded agent implementation / 24 hours
|   |   |-- Summary: Implement one auditable LangGraph workflow that cannot exceed its agreed authority.
|   |   |-- Build one bounded multi-step tool flow rather than a general multi-agent platform.
|   |   |-- Reuse Project 2 retrieval instead of rebuilding an advanced RAG stack.
|   |   |-- Add schema validation, step limits, token/request budgets, sandboxing, and a human checkpoint.
|   |   |-- Preserve an audit trail explaining tool calls, state transitions, failures, and human decisions.
|   |   |-- PEFT/LoRA fine-tuning is an optional specialization: it may be attempted in a scheduled GPU session, but is not
|   |       |-- a pass/fail graduation requirement because the common FDE core is deployment, evaluation, and delivery.
|   |
|   |-- Task 5.4: Agent evaluation and observability / 18 hours
|   |   |-- Summary: Prove useful behavior and expose important failures without relying on demo impressions.
|   |   |-- Build a compact golden scenario set covering success, refusal, escalation, and recovery.
|   |   |-- Run deterministic CI smoke evaluations with cached fixtures.
|   |   |-- Use Ragas for the bounded manual retrieval and response-quality evaluation path.
|   |   |-- Use Arize Phoenix to inspect traces, latency, cost, tool behavior, and user-visible failure.
|   |   |-- Calibrate a bounded manual LLM-as-judge sample instead of making it the CI authority.
|   |
|   |-- Task 5.5: Change control and dual-room defense / 16 hours
|   |   |-- Summary: Revise the solution under pressure and defend it to both technical and business audiences.
|   |   |-- Process a late stakeholder change through explicit impact analysis and approval.
|   |   |-- Demonstrate the architecture, safety boundaries, eval gaps, and rejected alternatives to engineers.
|   |   |-- Present the value case, limitations, adoption risk, and next investment decision to a simulated VP.
|   |   |-- Record the agreed change, deferred work, and remaining uncertainty.
|   |
+-- Project 6: AWS Capstone, Adoption and Handback
|   |-- Schedule: Weeks 20-22 / 60 hours
|   |-- Engagement: Take the locally accepted AI system through protected deployment, client acceptance, adoption planning, and responsible teardown.
|   |
|   |-- Task 6.1: Capstone kickoff and deployment agreement / 8 hours
|   |   |-- Summary: Agree on deployment responsibilities, acceptance evidence, cost limits, and exit conditions before using AWS.
|   |   |-- Conduct a deployment-readiness meeting covering security, support, data, access, and decision owners.
|   |   |-- Write a delivery plan with milestones, responsibilities, dependencies, and escalation paths.
|   |   |-- Confirm the $20 approved-API cap, $180 AWS cap, 14-day assessment window, and teardown contract.
|   |   |-- Define client acceptance criteria and a rollback/no-go threshold.
|   |
|   |-- Task 6.2: Protected AWS deployment / 18 hours
|   |   |-- Summary: Deploy the locally accepted stack through the supplied course-managed AWS path.
|   |   |-- Use the provided launch templates and one dedicated program-managed sandbox.
|   |   |-- Configure trusted TLS, reviewer authentication, request limits, rate limits, and secrets outside Git.
|   |   |-- Enforce IMDSv2, least-privilege permissions, controlled egress, synthetic evidence data, and bounded retention.
|   |   |-- Use scheduled GPU access only for approved model-serving evidence; no always-on GPU is permitted.
|   |   |-- Avoid turning the capstone into a general AWS, Terraform, or Kubernetes implementation course.
|   |
|   |-- Task 6.3: Acceptance, operations, and cost evidence / 12 hours
|   |   |-- Summary: Demonstrate that the deployed system meets its technical and business acceptance conditions.
|   |   |-- Run the course verifier and held-out functional, security, and resilience scenarios.
|   |   |-- Show quality, latency, failure, and cost evidence against the agreed criteria.
|   |   |-- Triage one simulated client acceptance failure and communicate its delivery impact.
|   |   |-- Keep remediation inside the agreed endpoint, assessment window, and budget.
|   |
|   |-- Task 6.4: Adoption and operational handback / 8 hours
|   |   |-- Summary: Prepare the client organization to operate, govern, and appropriately use the delivered system.
|   |   |-- Identify user groups, workflow changes, adoption risks, and responsible owners.
|   |   |-- Produce an operator runbook, user guidance, escalation map, and known-limitations register.
|   |   |-- Facilitate a simulated enablement session with resistant or skeptical users.
|   |   |-- Define post-launch measures for usage, value, quality, and safety.
|   |
|   |-- Task 6.5: Technical and business defenses / 8 hours
|   |   |-- Summary: Defend the completed engagement in two rooms using the evidence each audience needs.
|   |   |-- Run a technical defense covering architecture, controls, evals, operations, cost, and residual risks.
|   |   |-- Run an executive defense covering value achieved, adoption status, limitations, and next investment.
|   |   |-- Respond live to changed constraints without inventing evidence or promising unapproved scope.
|   |   |-- Obtain a simulated accept, conditional-accept, or reject decision with recorded rationale.
|   |
|   |-- Task 6.6: Teardown and reusable field playbook / 6 hours
|   |   |-- Summary: Close the engagement safely and turn customer-specific learning into reusable delivery knowledge.
|   |   |-- Execute and verify teardown of instances, volumes, public IPv4s, tokens, parameters, traces, images, and endpoint records.
|   |   |-- Deliver the final cost, teardown, acceptance, and unresolved-risk evidence.
|   |   |-- Separate reusable patterns from client-specific details and sensitive information.
|   |   |-- Publish a concise discovery-to-handback playbook for the next engagement.
|   |   |-- Record what should feed back into product, curriculum, or delivery tooling.
|   |
+-- Workload verification
|   |-- Project 1: 3 weeks x 20 hours = 60 hours
|   |-- Project 2: 4 weeks x 20 hours = 80 hours
|   |-- Project 3: 4 weeks x 20 hours = 80 hours
|   |-- Project 4: 4 weeks x 20 hours = 80 hours
|   |-- Project 5: 4 weeks x 20 hours = 80 hours
|   |-- Project 6: 3 weeks x 20 hours = 60 hours
|   |-- Program: 22 weeks x 20 hours = 440 hours
```